LAPORAN PRAKTIKUM & TUGAS PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
ABSTRACTION

Dosen Pengampu :

Afdhal Dinilhak M.Kom

Asisten Praktikum :

Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

Identitas

Nama : Razif Al Farisi

Nim : 2511532028

Kelas : B

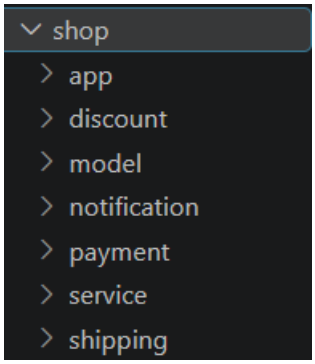
NimParams :

```
Parameter tugas untuk NIM 2511532028
  CARD_ADMIN_PERCENT  = 3
  TRANSFER_ADMIN_FEE  = 4000
  SMS_MAX_LENGTH      = 128
PS D:\praktikum-oop\TUGAS_2511532028_RazifAlFarisi\abstraction-2511532028>
```

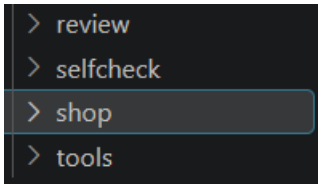
LATIHAN PRAKTIKUM 6

A. L0 Menyiapkan direktori proyek baru

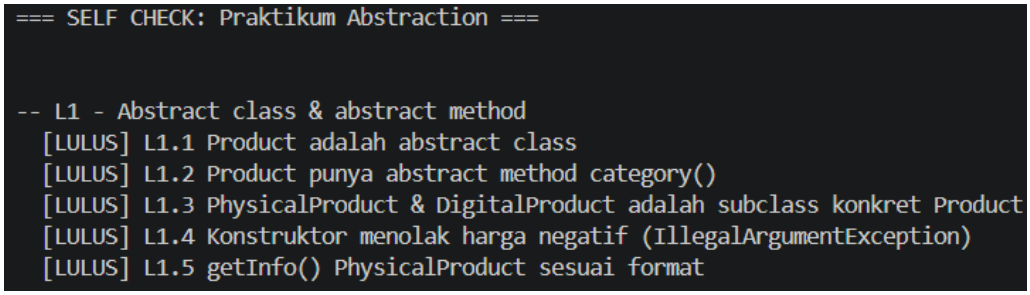
1. Membuat sebuah projek java baru dengan ketik **“Java: Create Java Project...,”** dan pilih No build tools. Hapus App.Java



2. Setelah itu buat sebuah package java dengan nama “shop.model”, “shop.shipping”, “shop.discount”, dan lainnya(titik pada nama tersebut lansung membuat nested package. Maksudnya di dalam package shop ada package model)



3. Setelah itu salin alat bantu yang ada di file .zip yang diberikan saat praktikum. Letakkan di dalam src



4. SelfCheck digunakan untuk memeriksa kesesuaian kode & hasil dari program dengan modul praktikum yang diberikan.

B. L1 : Abstract Class dan Abstract Method

1. Membuat Sebuah Class dengan nama Product.java. pada package model

```java
package shop.model;

public abstract class Product {

    private final String sku;
    private final String name;
    private final long price;

    protected Product(String sku, String name, long price) {
        if (sku == null || sku.isBlank()) {
            throw new IllegalArgumentException("SKU must not be blank");
        }
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        if (price < 0) {
            throw new IllegalArgumentException("Price must not be negative");
        }
        this.sku = sku;
        this.name = name;
        this.price = price;
    }

    public String getSku() { return sku; }
    public String getName() { return name; }
    public long getPrice() { return price; }
```

```java
    /** Price the customer actually pays. Subclasses may override it */
    public long getFinalPrice() { return price; }

    /** Every concrete product must state its category */
    public abstract String category();

    /** A concrete method that relies on the abstract method category */
    public String getInfo() {
        return "[" + category() + "] " + name + " - Rp" + getFinalPrice();
    }
}
```

class ini yang akan menjadi objek untuk menampilkan dan penyedia produk-produk yang dijual. Atribut diberikan hak akses ***private*** agar menghindari seseorang memanipulasi / mengubah data tersebut. Pengguna hanya bisa mengambil / melihat nilai dari atribut tersebut dengan method ***getter***. Method category akan diinisialisasi oleh subclass agar method ***getInfo()*** mengetahui produk tersebut termasuk ke kategori mana.

2. Membuat class PhysicalProduct.java

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {

    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be positive");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() { return weightGrams; }

    @Override
    public String category() {
        return "Fisik";
    }

}
```

Class ini berisikan constructor untuk membuat objek produk yang bersifat fisik yang harus memiliki berat. Produk memiliki berat tersebut akan di kategorikan sebagai “Fisik” dan atributnya bersifat private yang hanya bisa dilihat / diambil melalui ***getWeightGrams()***.

3. Membuat Class DigitalProduct.java

```java
package shop.model;

import shop.discount.Discountable;

public class DigitalProduct extends Product implements Discountable{

    private String downloadUrl;
    private final int DIGITAL_DISCOUNT_PERCENT = 10;

    public DigitalProduct(String sku, String name, long price, String downloadUrl) {
        super(sku, name, price);
        if (downloadUrl == null || downloadUrl.isBlank()) {
            throw new IllegalArgumentException("Download URL must not be blank");
        }
        this.downloadUrl = downloadUrl;
    }

    public String getDownloadUrl() {
        return downloadUrl;
    }

    @Override
    public String category() {
        return "Digital";
    }
```

```
    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }

}
```

Class ini untuk objek produk yang bersifat digital yang tidak memiliki berat dan tidak bisa diraba. Produk tersebut akan diberikan tag kategori "Digital" dan harus memiliki URL untuk bisa akses / mengunduh produk tersebut.

4. Membuat Class ShippingService.java pada package shipping

```
package shop.shipping;

public abstract class ShippingService {

    private final String name;

    protected ShippingService(String name) {
        if (name == null || name.isBlank()) {
            throw new IllegalArgumentException("Name must not be blank");
        }
        this.name = name;
    }

    public String getName() { return name;}

    /** rate in rupiah per started kilogram */
    protected abstract long ratePerKg();

    /** weight is rounded up: 700 g -> 1 kg, 1001 g -> 2kg. */
    public final long calculateCost(int weightGrams) {
        if (weightGrams <= 0) {
            return 0;
        }
        long kilograms = (weightGrams + 999L) / 1000;
        return kilograms * ratePerKg();
    }
```

Ini adalah class abstract untuk semua kurir, yang membedakan hanya tarif per kilogram. Terdapat method yang tidak boleh diubah-ubah (final), yaitu ***calculateCost()***. Method tersebut harus memeliki berat lebih dari 0 dan berat tersebut akan ditambah dan dibagi dengan 1000. Lalu method mengembalikan nilai kilograms yang dikalikan dengan method ***ratePerKg()***.

5. Class RegularShipping.java & ExpressShipping.java

```java
package shop.shipping;

public class RegularShipping extends ShippingService {

    public RegularShipping() {
        super(name: "Regular");
    }

    @Override
    protected long ratePerKg() {
        return 10_000;
    }
}
```

```java
package shop.shipping;

public class ExpressShipping extends ShippingService {

    public ExpressShipping() {
        super(name: "Express");
    }

    @Override
    protected long ratePerKg() {
        return 18_000;
    }

}
```

Kedua class tersebut isinya sama dan sama-sama extends dari class **ShippingService.java**, hanya isi dari methodnya yang berbeda. Pada class **RegularShipping.java**, kurirnya diberikan tag kategori "Regular" dan ratePerKilogramnya adalah 10000. Untuk class **ExpressShipping.java**, objek kurir dikategorikan "Express" dan memiliki ratePerKilogramnya sebesar 18000.

6. Percobaan membuat objek Product

```
The type new Product(){} must implement the inherited abstract method Product.category() Java(67109264) [Ln 28, Col 25]
```

Saat mencoba membuat objek "Product p = new Product("x","x",1000);". Sistem mengirimkan pesan error untuk membuat objek tersebut melalui subclass / class yang mewariskan class product tersebut, karena product memiliki method abstract / masih kosong yang harus dioverride terlebih dahulu.

C. L2: Interface dan Program to an Interface

1. Membuat Shippable.java pada package shipping

```java
package shop.shipping;

public interface Shippable {

    int getWeightGrams();

}
```

```java
package shop.model;

import shop.shipping.Shippable;

public class PhysicalProduct extends Product implements Shippable {

    private final int weightGrams;

    public PhysicalProduct(String sku, String name, long price, int weightGrams) {
        super(sku, name, price);
        if (weightGrams <= 0) {
            throw new IllegalArgumentException("Weight must be positive");
        }
        this.weightGrams = weightGrams;
    }

    @Override
    public int getWeightGrams() { return weightGrams; }

    @Override
    public String category() {
        return "Fisik";
    }
```

Membuat sebuah interface / kontrak yang hanya berisi method ***getWeightGrams()***. Class yang mengimplement interface ini wajib meng-*override* method tersebut. Pada class **PhysicalProduct.java,** implement **Shippable** dan method ***getWeightGrams()*** dengan mengembalikan nilai weightGrams yang ada pada class tersebut. Untuk **DigitalProduct** tidak menggunakan / implement **Shippable** ini, karena productnya tidak dikirim dan bisa digunakan / diakses melalu URL.

2. Interface PaymentMethod.java

```java
package shop.payment;

public interface PaymentMethod {

    String getName();

    /** @return true if the payment succeeds */
    boolean pay(long amount);

    /* T1.a  */
    // abstract long adminFee(long amount);
    default long adminFee(long amount) {
        return 0;
    }
}
```

Class untuk pembayaran product yang ingin dibeli. Method ***pay()*** hanya mengembalikan nilai boolean yang menandakan pembayaran berhasil atau gagal.

3. Class EwalletPayment.java

```java
package shop.payment;

public class EWalletPayment implements RefundablePayment {

    private final String provider;
    private long balance;

    public EWalletPayment(String provider, long balance) {
        if (provider == null || provider.isBlank()) {
            throw new IllegalArgumentException("Provider must not be blank");
        }
        if (balance < 0) {
            throw new IllegalArgumentException("Balance must not be negative");
        }
        this.provider = provider;
        this.balance = balance;
    }

    public long getBalance() { return balance; }

    @Override
    public String getName() { return "E-Wallet " + provider + getBalance(); }
```

```java
    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > balance) {
            return false;
        }
        this.balance -= amount;
        return true;
    }

    @Override
    public boolean refund(long amount) {
        if (amount <= 0) {
            return false;
        }
        this.balance += amount;
        return true;
    }
}
```

Terdapat dua atribut, yaitu provider dan balance. Class ini mengimplement interface **PaymentMethod** yang harus meng-*override* method method pada interface tersebut. Method ***pay()*** mengurangi balance pada class ini dan mengembalikan nilai true yang menandakan berhasil dibayar.

4. Class BankTransferPayment.java

```java
package shop.payment;

import shop.StudentInfo;

public class BankTransferPayment implements PaymentMethod {

    private String bankName;

    public BankTransferPayment(String bankName) {
        this.bankName = bankName;
    }

    @Override
    public String getName() {
        return "Transfer " + bankName;
    }

    @Override
    public boolean pay(long amount) {
        return amount > 0;
    }
```

Class **BankTransferPayment** yang mewarisi method dari **PaymentMethod**. Class ini hanya menyimpan nama bank. Methodnya ***pay()*** mengembalikan nilai true jika harga yang harus dibayar besar dari 0.

5. Class CheckoutService.java pada package service

```java
package shop.service;

import java.util.List;
import shop.model.Product;
import shop.payment.PaymentMethod;
import shop.payment.RefundablePayment;
import shop.shipping.Shippable;
import shop.shipping.ShippingService;


public class CheckoutService {

    public long subtotal(List<Product> items) {
        long total = 0;
        for (Product product : items) {
            total += product.getFinalPrice();
        }
        return total;
    }

    public int totalWeight(List<Product> items) {
        int grams = 0;
        for (Product product : items) {
            if (product instanceof Shippable shippable) { // a capability, not a class
                grams += shippable.getWeightGrams();
            }
        }
        return grams;
    }

    public long total(List<Product> items, ShippingService shipping) {
        return subtotal(items) + shipping.calculateCost(totalWeight(items));
    }

    public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
        if (items == null || items.isEmpty()) {
            return false;
        }
        payment.pay(total(items, shipping));
        return true;
    }

    public boolean refund(RefundablePayment payment, long amount) {
        if (payment == null || amount <= 0) {
            return false;
        }
        payment.refund(amount);
        return true;
    }
```

Class ini yang akan melakukan checkout / pembayaran. Terdapat method ***subtotal()*** dan ***total()***, method ***totalWeight()*** mengembalikan nilai gram jika produk mewarisi sifat dari class **Shippable**. Lalu, barang-barang yang sudah ditotalkan akan di bayar dengan memanggil method **payment.pay()** dan mengembalikan **true**.

6. Membuat Class ShopApp.java (main)

```java
package shop.app;

import java.util.List;

import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;

import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;
import shop.shipping.RegularShipping;
import shop.shipping.ShippingService;
import shop.payment.EWalletPayment;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.PaymentMethod;


public class ShopApp {
    Run | Debug
    public static void main(String[] args) {
        // L1 => abstract class dan abstract method
        Product coffee = new PhysicalProduct(sku: "KOPI-001", name: "Kopi Arabika Gayo 250g", price: 85_000, weightGrams: 300);

        Product rendang = new PhysicalProduct(sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400);
```

```java
        Product rendang = new PhysicalProduct(sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400);

        Product ebook = new DigitalProduct(sku: "EBK-003", name: "E-book Resep Kue Nusantara", price: 50_000, downloadUrl: "https://toko-

        List<Product> cart = List.of(coffee, rendang, ebook);
        System.out.println("== Katalog ==");
        for (Product product : cart) {
            System.out.println(product.getInfo());
        }

        // L2 => interface and program to an interface
        CheckoutService  checkout = new CheckoutService();
        ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();


        System.out.println("\n== Ringkasan ==");
        System.out.println("Subtotal        : Rp" + checkout.subtotal(cart));
        System.out.println("Berat kiriman   : " + checkout.totalWeight(cart) + " g");
        System.out.println("Total Reguler   : Rp" + checkout.total(cart, regular));
        System.out.println("Total Express   : Rp" + checkout.total(cart, express));

        // L3.e
        EWalletPayment wallet = new EWalletPayment(provider: "DompetKu : ", balance: 300_000);
```

```
== Katalog ==
[Fisik] Kopi Arabika Gayo 250g - Rp85000
[Fisik] Rendang Kemasan 250g - Rp120000
[Digital] E-book Resep Kue Nusantara - Rp45000

== Ringkasan ==
Subtotal        : Rp250000
Berat kiriman   : 700 g
Total Reguler   : Rp260000
Total Express   : Rp268000
```

Semua class yang sudah di buat diimport ke ShopApp. Objek di buat dengan nama product coffee, rendang, dan e-book. Lalu, objek keranjang dibuat menggunakan List<>, product yang ada dalam keranjang akan ditampilkan info detailnya. Kemudian objek checkout dan kurir regular serta express dibuat. Subtotal product, total berat yang akan dirikim, dan total dengan kurir regular / express di tampilkan.

D. L3: Interface Lanjutan(default, static, private method dan interface inheritance)

1. Class Discountable.java pada package discount

```java
package shop.discount;

public interface Discountable {

    int discountPercent(); // abstract: each class decides

    default long applyDiscount(long price) {
        return price - percentOf(price, safePercent());
    }

    static long percentOf(long amount, int percent) {
        return amount * percent / 100;
    }

    private int safePercent() {
        int percent = discountPercent();
        if (percent < 0) {
            return 0;
        }
        return Math.min(percent, 100);
    }
```

Interface ini memberikan discount untuk produk-product tersebut. Method default **applyDiscount()** akan mewarisi ke subclass yang akan mengimplement interface ini. Method static **percentOf()** yang akan mengubah nilai diskon menjadi persentase(%). Method default bisa di-*override* tapi method static tidak bisa di-override. Method **math.min()** itu berfungsi untuk mencari nilai terkecil dari parameter yang diberikan(percent, 100).

2. Memberi diskon pada class DigitalProduct.java

```java
public class DigitalProduct extends Product implements Discountable{

    private String downloadUrl;
    private final int DIGITAL_DISCOUNT_PERCENT = 10;
```

```java
    @Override
    public int discountPercent() {
        return DIGITAL_DISCOUNT_PERCENT;
    }

    @Override
    public long getFinalPrice() {
        return applyDiscount(getPrice());
    }
```

DigitalProduct ini extends product sekaligus mengimplement **Discountable**. Diskon pada product ini sebesar 10, lalu diskon tersebut dijadikan persentase melalui method **discountPercent()** dan diterapkan pada harga dengan method **getFinalPrice()**.

3. Pewarisan Interface ke Interface lain

```java
package shop.payment;

public interface RefundablePayment extends PaymentMethod{

    boolean refund(long amount);

    // T1.e
    default String policy() {
        return "Refund maksimal 7 hari";
    }

}
```

```java
package shop.payment;

public interface InstallmentPayment extends PaymentMethod {

    int maxInstallmentMonths();

    /** Monthly amount, rounded up */
    default long monthlyInstallment(long amount, int months) {
        if (months < 1 || months > maxInstallmentMonths()) {
            throw new IllegalArgumentException
                ("Months must be between 1 and " + maxInstallmentMonths());
        }
        return (amount + months - 1) / months; // Round up
    }

    // T1.e
    default String policy() {
        return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
    }

}
```

Membuat dua interface dengan nama **RefundablePayment** dan **InstallmentPayment** yang sama mengextends **PaymentMethod**. **RefundablePayment** mempunyai method **refund()** dan untuk **InstallmentPayment** mempunyai attribut **maxInstallMonths** serta method **monthlyInstallment()** untuk berapa yang harus dicicil dalam 1 bulan.

4. Modifikasi EwalletPayment, CreditCardPayment, CheckoutService, dan ShopApp

```java
public class EWalletPayment implements RefundablePayment {
```

```java
@Override
public boolean refund(long amount) {
    if (amount <= 0) {
        return false;
    }
    this.balance += amount;
    return true;
}
```

Pada class **EwalletPayment**, ditambahkan implement **RefundablePayment** dan method **refund()**. Jika amount tersebut lebih dari 0(true), **balance** pada class ini akan dikembalikan / bertambah.

```java
public class CreditCardPayment implements RefundablePayment, InstallmentPayment {

    private long usedCredit;
    private String cardHolder;
    private long creditLimit;

    public CreditCardPayment(String cardHolder, long creditLimit) {
        this.cardHolder = cardHolder;
        this.creditLimit = creditLimit;
    }

    @Override
    public String getName() {
        return "Kartu Kredit (" + cardHolder + ")";
    }

    public boolean pay(long amount) {
        if (amount <= 0 || usedCredit + amount > creditLimit) {
            return false;
        }
        usedCredit += amount;
        return true;
    }
```

```java
    @Override
    public boolean refund(long amount) {
        if (amount <= 0 || amount > usedCredit) {
            return false;
        }
        usedCredit -= amount;
        return true;
    }

    @Override
    public int maxInstallmentMonths() {
        return 12;
    }

    public long getUsedCredit() {
        return usedCredit;
    }

    // T1.c
    @Override
    public long adminFee(long amount) {
        return (amount * StudentInfo.CARD_ADMIN_PERCENT) / 100;
    }

    @Override
    public String policy() {
        return RefundablePayment.super.policy() + "; " + InstallmentPayment.super.policy();
    }
```

Membuat class **CreditCardPayment** yang mengimplement dua interface (**RefundablePayment & InstallmentPayment**). Terdapat sebuah konstruktor untuk mendaftarkan pembayaran dengan nama pemegang kartu dan batas penggunaan.

```java
public boolean refund(RefundablePayment payment, long amount) {
    if (payment == null || amount <= 0) {
        return false;
    }
    payment.refund(amount);
    return true;
}
```

Pada class **CheckoutService**, method refund di-*override* yang dimana payment yang digunakan mendukung fitur refund dan jumlahnya tidak

boleh lebih kecil dari 0. Lalu **paymeny.refund()** dijalankan jika kondisi bernilai true.

```
The method refund(RefundablePayment, long) in the type CheckoutService is not applicable for the arguments (BankTransferPayment, int) Java(67108979) [Ln 75, Col 18]
```

Saat dicoba melakukan refund pada payment dengan metode pembayaran bank, sistem menjadi error. Karena **pembayaran dengan bank** tidak mendukung fitur refund(method **refund()** tidak ada di **BankTransferPayment**).

5. Penambahan pada ShopApp.java

```java
// L3.e
EWalletPayment wallet = new EWalletPayment(provider: "DompetKu : ", balance: 300_000);
PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");
System.out.println("\n== Pembayaran ==");
System.out.println(wallet.getName() + " berhasil? " + checkout.checkout(cart, regular,
wallet));
System.out.println("Sisa saldo: Rp" + wallet.getBalance());
System.out.println(transfer.getName() + " berhasil? " + checkout.checkout(cart, express,
transfer));

// L3: interface inheritance, multiple interfaces, default method
CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);
System.out.println(card.getName() + " berhasil? " + checkout.checkout(cart, express, card));
System.out.println("Cicilan 3 bulan: Rp" + card.monthlyInstallment(checkout.total(cart,
express), months: 3));

System.out.println("\n== Refund e-book ==");
System.out.println("Refund e-wallet berhasil? " + checkout.refund(wallet,
ebook.getFinalPrice()));

System.out.println("Saldo setelah refund: Rp" + wallet.getBalance());

// L3.d bagian 4
/**
 * method refund tidak ada dan tidak bisa di aplikasikan ke objek.
 */
```

ShopApp mengaplikasikan fitur refund dan cicilan, hasillnya seperti berikut :

```
== Pembayaran ==
E-Wallet DompetKu : 300000 berhasil? true
Sisa saldo: Rp40000
Transfer Bank Nagari berhasil? true
Kartu Kredit (Siti Rahma) berhasil? true
Cicilan 3 bulan: Rp89334

== Refund e-book ==
Refund e-wallet berhasil? true
Saldo setelah refund: Rp85000
```

E. L4: Final Method dan Final Class

1. Modifikasi method pada class ShippingService.java

```java
public final long calculateCost(int weightGrams) {
    if (weightGrams <= 0) {
        return 0;
    }
    long kilograms = (weightGrams + 999L) / 1000;
    return kilograms * ratePerKg();
}
```

Method ini tidak dapat di-Override oleh subclass, dilakukan Uji coba menggunakan class sementara **FreeShipping extends ShippingService** yang mengoverride method tersebut.

```java
package shop.shipping;

public class FreeShipping extends ShippingService {

    protected FreeShipping(String name) {
        super(name);
        //TODO Auto-generated constructor stub
    }

    @Override
    public final long calculateCost(int weightGrams) {

    }

    @Override
    protected long ratePerKg() {
        // TODO Auto-generated method stub
        throw new UnsupportedOperationException("Unimplemented method 'ratePerKg'");
    }

}
```

```
⊗ Cannot override the final method from ShippingService Java(67109265) [Ln 11, Col 23]
```

Sistem memberitahu bahwa method final pada **ShippingService** tidak bisa di-override.

2. Membuat class Address.java di package model

```java
package shop.model;

public final class Address {
    private final String street;
    private final String city;
    private final String postalCode;

    public Address(String street, String city, String postalCode) {
        if (street == null || street.isBlank()) {
            throw new IllegalArgumentException("Street must not be blank");
        }
        if (city == null || city.isBlank()) {
            throw new IllegalArgumentException("City must not be blank");
        }
        if (postalCode == null || postalCode.isBlank()) {
            throw new IllegalArgumentException("Postal code must not be blank");
        }
        this.street = street;
        this.city = city;
        this.postalCode = postalCode;
    }

    public String getStreet() { return street; }
    public String getCity() { return city; }
    public String getPostalCode() { return postalCode; }

```

```
    public String getStreet() { return street; }
    public String getCity() { return city; }
    public String getPostalCode() { return postalCode; }

    @Override
    public String toString() {
        return street + ", " + city + " " + postalCode;
    }
}
```

Class ini berfungsi untuk menampilkan alamat dari pembeli berupa jalan, kota, dan kodepostal. Method **toString()** untuk menampilkan format informasi mengenai alamat tersebut.

F. L5: Nested Class(static nested, inner, dan anonymous class)

1. Membuat class Order.java di package model

```
package shop.model;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Order {
    private final String orderId;
    private final Address shippingAddress;
    private final List<Line> lines = new ArrayList<>();

    public Order(String orderId, Address shippingAddress) {
        if (orderId == null || orderId.isBlank() || shippingAddress == null) {
            throw new IllegalArgumentException("Order id and address are required");
        }
        this.orderId = orderId;
        this.shippingAddress = shippingAddress;
    }

    public String getOrderId() { return orderId; }
    public Address getShippingAddress() { return shippingAddress; }

    public void addLine(Product product, int quantity) {
        lines.add(new Line(product, quantity));
    }

    public List<Line> getLines() { return Collections.unmodifiableList(lines); }
```

Class ini berfungsi untuk menyimpan baris pesanan yang terdiri dari **orderid,** dan **shippingAddress**. Atribut tersebut harus memiliki nilai agar objek untuk baris pesanan dapat dibuat. Method **addLines**() untuk menambahkan product yang diorder ke dalam list.

2. Lanjutan class Order.java

```java
    public List<Line> getLines() { return Collections.unmodifiableList(lines); }


    public static class Line {

        private final Product product;
        private final int quantity;


        public Line(Product product, int quantity) {
            if (product == null || quantity <= 0) {
                throw new IllegalArgumentException("Product must not be null");
            }
            this.product = product;
            this.quantity = quantity;
        }

        public Product getProduct() { return product; }
        public int getQuantity() { return quantity; }

        public long subtotal() {
            return product.getFinalPrice() * quantity;
        }

    }
```

```java
}

public long getItemsTotal() {
    long total = 0;
    for (Line line : lines) {
        total += line.subtotal();
    }
    return total;
}

public class Receipt {

    public String toText() {
        StringBuilder sb = new StringBuilder();

        // Format Baris Pertama: STRUK PESANAN <orderId>
        sb.append("STRUK PESANAN ").append(Order.this.orderId).append("\n");

        // Satu baris per item: - <nama> x<qty> = Rp<subtotal>
        for (Line line : lines) {
            sb.append("- ")
              .append(line.getProduct().getName()) // Mengambil nama produk
              .append(" x").append(line.getQuantity())
              .append(" = Rp").append(line.subtotal())
              .append("\n");
        }
```

```
        for (Line line : lines) {
            sb.append("- ")
              .append(line.getProduct().getName()) // Mengambil nama produk
              .append(" x").append(line.getQuantity())
              .append(" = Rp").append(line.subtotal())
              .append("\n");
        }

        // Total pesanan: Rp<total>
        sb.append("Total pesanan: Rp").append(getItemsTotal()).append("\n");

        // Kirim ke: <alamat>
        sb.append("Kirim ke: ").append(Order.this.shippingAddress);

        return sb.toString();
    }
}

public Receipt createReceipt() {
    return new Receipt();
}
```

Terdapat 2 class di dalam class **Order.java** ini, yaitu **Line** dan **Receipt**. Class **Line** ini yang mengurus satu baris item yang berisikan **product** dan **quantity,** product tersebut akan dijumlahkan menggunakan method **subtotal()** untuk per baris pesanan. Class **Receipt** bertugas untuk mencetak struk dari pesanan tersebut. Penyusunan teks struk tersebut menggunakan **StringBuilder** bawaan dari java dan menambahkan ke dalam **StringBuilder** dengan perintah **.append()**.

3. Melengkapi ShopApp.java

```
Address address = new Address(street: "Jl. Moh Hatta No. 10", city: "Padang", postalCode: "25121");
Order order = new Order(orderId: "ORD-0001", address);
order.addLine(coffee, quantity: 2);
order.addLine(ebook, quantity: 1);
Order.Receipt receipt = order.new Receipt(); // inner object needs an outer object
System.out.println(receipt.toText());

PaymentMethod voucher = new PaymentMethod() { // anonymous class
    private long remaining = 50_000;

    @Override
    public String getName() { return "Voucher Promo Hari Ini"; }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 || amount > remaining) {
            return false;
        }
        remaining -= amount;
        return true;
    }
};
    System.out.println("\n" + voucher.getName() + " untuk e-book berhasil? "
    + checkout.checkout(List.of(ebook), regular, voucher));
```

Objek **address dan order** ditambahkan, product yang sudah dipesan tersebut di tambahkan ke dalam **order** dan struk di cetak. Sebuah anonymous class di tambahkan dengan nama voucher dengan attribur remaining = 50000. Kemudian berhasil / tidak dari voucher tersebut dicetak.

4. Bagian SelfCheck dari L1 hingga L5

```
-- L5 - Static nested, inner, dan anonymous class
  [LULUS] L5.1 Order.Line static nested; Order.Receipt inner (non-static)
  [LULUS] L5.2 Receipt.toText() memuat id pesanan, total barang, dan kota tujuan
  [LULUS] L5.3 ShopApp memakai anonymous class yang mengimplementasikan PaymentMethod

=== HASIL: 23 / 23 pemeriksaan lulus ===
PS D:\praktikum-oop\TUGAS_2511532028_RazifAlFarisi\abstraction-2511532028>
```

Semua tahapan berhasil di terapkan

TUGAS PRAKTIKUM 6

A. T0: Parameter Pribadi & NIM

1. Menjalankan perintah “java -cp bin tools.NimParams Nim(2511532028)”

```
Parameter tugas untuk NIM 2511532028
  CARD_ADMIN_PERCENT  = 3
  TRANSFER_ADMIN_FEE  = 4000
  SMS_MAX_LENGTH      = 128
```

2. Setelah itu menyalin ketiga nilai yang di cetak ke **StudentInfo.java**

```java
public final class StudentInfo {
    public static final String NIM = "2511532028";
    public static final String NAME = "Razif Al Farisi";

    public static final int CARD_ADMIN_PERCENT = 3;
    public static final long TRANSFER_ADMIN_FEE = 4000;
    public static final int SMS_MAX_LENGTH = 128;

    private StudentInfo() {
        // constants only; no objects
    }
}
```

B. T1: Evolusi API dan konflik diamond

1. Menambahkan method adminFee() ke PaymentMethod.java

```
ShopApp.java src\shop\app 1
  The type new PaymentMethod(){} must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 84, Col 37]
EWalletPayment.java src\shop\payment 1
  The type EWalletPayment must implement the inherited abstract method PaymentMethod.adminFee(long) Java(67109264) [Ln 3, Col 14]
```

Masalah ini terjadi karena sebuah class **PaymentMethod** yang sudah digunakan pada app yang sudah jadi, maka semua class yang mengimplement **PaymentMethod** dipaksa mengisi method yang kosong tersebut(termasuk anonymous class pada ShopApp). Class yang mengalami error ada **ShopApp** dan **EwalletPayment**.

2. Menambahkan Default Method

```
    /* T1.a  */
    // abstract long adminFee(long amount);
    default long adminFee(long amount) {
        return 0;
    }
}
```

Method ini mencegah error yang terjadi pada T1 sebelumnya. Dengan menambahkan Default, class yang mengimplement interface ini, namun tidak mengisi / *override* method tersebut tidak akan mengalami error, karena class tersebut otomatis mengambil nilai sifat bawaan dari interface yang sudah menerapkan kode "default".

3. Meng-Override class CreditCardPayment, BankTransferPayment, dan EwalletPayment

```
public class CreditCardPayment implements RefundablePayment, InstallmentPayment {
    // T1.c
    @Override
    public long adminFee(long amount) {
        return (amount * StudentInfo.CARD_ADMIN_PERCENT) / 100;
    }
```

CreditCardPayment mengisi method tersebut dengan jumlah yang dikalikan dengan biaya admin(persen) yang di dapat pada class **StudentInfo** dan dibagi 100. Untuk EWalletPayment tidak dikenai biaya admin.

```
public class BankTransferPayment implements PaymentMethod {
    // T1.c
    @Override
    public long adminFee(long amount) {
        return StudentInfo.TRANSFER_ADMIN_FEE;
    }

}
```

BankTransferPayment meng-*override* method tersebut dengan mengembalikan nilai biaya transfer admin pada class **StudentInfo**.

4. Menambahkan method di CheckoutService.java

```
// T1.d
public long grandTotal(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    return total(items, shipping) + payment.adminFee(total(items,shipping));
}
```

Menambahkan Method **grandTotal()**, yaitu total yang harus dibayar ditambahkan dengan biaya admin yang di kenakan.

```
public boolean checkout(List<Product> items, ShippingService shipping, PaymentMethod payment) {
    if (items == null || items.isEmpty()) {
        return false;
    }
    payment.pay(grandTotal(items, shipping, payment));
    return true;
}
```

Lalu, pada method **checkout()** bagian **payment.pay(total())** diubah menjadi **payment.pay(grandTotal()).**

5. Konflik diamond

```
// T1.e
default String policy() {
    return "Cicilan maksimal " + maxInstallmentMonths() + " bulan";
}
```

```
// T1.e
default String policy() {
    return "Refund maksimal 7 hari";
}
```

Menambahkan dua method default yang sama ke dua interface.

```
Duplicate default methods named policy with the parameters () and () are inherited from the types InstallmentPayment and RefundablePayment Java(67109917) [Ln 5, Col 14]
```

Sistem error pada class **CreditCardPayment**, karena class tersebut mengimplement kedua interface yang diberi method **policy()**. Jadi sistem menganggap terdapat sifat / method yang duplicate.

- **EWalletPayment** hanya mengimplement salah satu, yaitu **RefundablePayment**. Jadi tidak ada masalah pada class ini.
- **Java** ini memiliki desain / aturan yang ketat. Kalau dibiarkan java memilih / menebak, sistem bisa saja mengalami bug yang sulit dilacak.

6.

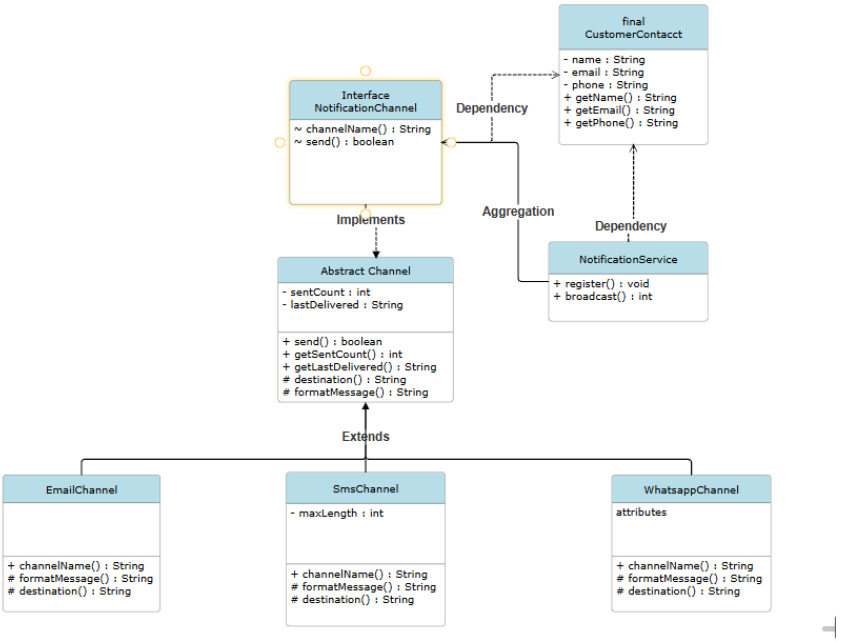
```
NIM: 2511532028 | Razif Al Farisi

-- Parameter NIM
  [LULUS] P.1 Konstanta di StudentInfo sesuai NIM (3%, Rp4000, 128 karakter)

-- T1 - Evolusi API & diamond
  [LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
  [LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
  [LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
  [LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
```

G. T2: Merancang Sistem Notifikasi Pesanan

1. Diagram UML



2. Membuat berbagai class di package notification

```java
package shop.notification;

public final class CustomerContact {

    private String name;
    private String email;
    private String phone;

    public CustomerContact(String name, String email, String phone) {
        if (name == null) {
            throw new IllegalArgumentException("Name cannot be null");
        } else {
            this.name = name;
        }
        if (email == null) {
            this.email = "";
        } else {
            this.email = email;
        }
        if (phone == null) {
            this.phone = "";
        } else {
            this.phone = phone;
        }
    }

    public String getName() { return name;}
    public String getEmail() { return email;}
    public String getPhone() { return phone;}
```

CustomerContact digunakan untuk menyimpan informasi seseorang seperti nama, email, dan nomor telepon. Field nama tidak boleh kosong, untuk phone dan email diizinkan tidak diisii.

```java
package shop.notification;

public interface NotificationChannel {

    // T2
    String channelName();

    boolean send(CustomerContact contact, String message);

}
```

Sebuah interface yang mewajibkan subclass untuk meng-*override* method-method tersebut.

```java
package shop.notification;

public abstract class AbstractChannel implements NotificationChannel {

    // T2

    private int sentCount = 0;
    private String lastDelivered = null;

    public int getSentCount() { return sentCount; }
    public String getLastDelivered() { return lastDelivered; }

    @Override
    public final boolean send(CustomerContact contact, String message) {
        if (message == null || message.isEmpty()) {
            throw new IllegalArgumentException("Message cannot be null or empty!");
        }

        String destination = destination(contact);

        if (destination == null) {
            return false;
        } else  {

        String formattedMessage = formatMessage(contact, message);

        System.out.println(formattedMessage);
```

```java
        System.out.println(formattedMessage);

        sentCount++;
        lastDelivered = formattedMessage;

        return true;
        }
    }

    protected abstract String destination(CustomerContact contact);
    protected abstract String formatMessage(CustomerContact contact, String message);
```

Class abstract yang implement **NotificationChannel**, attributnya terdiri dari **sentCount(jumlah yang tekirim)** dan **lastDelivered(Pesan terakhir terkirim)**. Pada method **send()**, message tidak boleh kosong dan destinasi harus ada walaupun hanya " " agar pesan terkirim dan ditampilkan melalui **Sysout.println(formatMessage)**. Method destination, dan formatMessage akan diisi di subclass(Email, Sms, Whatsapp).

```java
package shop.notification;

public class EmailChannel extends AbstractChannel{

    public EmailChannel() {
    }

    public String channelName() {
        return "Email";
    }

    @Override
    public String destination(CustomerContact contact) {
        if (!contact.getEmail().contains("@")) {
            return null;
        }
        return contact.getEmail();
    }

    @Override
    protected String formatMessage(CustomerContact contact, String message) {

        return "[Email to " + destination(contact) + "] " + message;
    }

}
```

Pada kanal email, email seseorang harus memuat karakter "@" agar bisa dikirimkan ke email pembeli / pengguna dengan formatMessage().

```java
package shop.notification;

public class SmsChannel extends AbstractChannel {

    private int maxLength;

    public SmsChannel(int maxLength) {
        this.maxLength = maxLength;
    }

    @Override
    public String channelName() {
        return "SMS";
    }

    @Override
    protected String destination(CustomerContact contact) {
        if (!contact.getPhone().startsWith("08")) {
            return null;
        }
        return contact.getPhone();
    }

    @Override
    protected  String formatMessage(CustomerContact contact, String message) {
        if (message.length() > maxLength) {
            return "[SMS to " + destination(contact) + "] " + message.substring(0, maxLength) + "...";
        }
        return "[SMS] " + message;
    }
```

Kanal ini memiliki attribut **maxLength** yang tujuannya memberikan batas panjang pesanan tersebut. Pada method **formatMessage()**, terdapat logika if jika pesan tersebut lebih dari **maxLength yang di tentukan**, maka dari batas **maxLength** hingga pesan habis, pesan akan dipotong dan di ganti dengan "...".

```java
package shop.notification;

public class WhatsAppChannel extends AbstractChannel {

    public WhatsAppChannel() {

    }

    @Override
    public String channelName() {
        return "Whatsapp";
    }

    @Override
    protected String destination(CustomerContact contact) {
        if (contact.getPhone() == null || contact.getPhone().length() < 10) {
            return null;
        } else
        if (contact.getPhone().startsWith("08") || contact.getPhone().startsWith("+62")) {
            return contact.getPhone();
        }
        return null;
    }

    @Override
    protected String formatMessage(CustomerContact contact, String message) {
        return "[WA to "+ destination(contact) + "] [Toko UMKM] " + message;
    }
```

Kanal ini mengirimkan pesan melalui nomor **Whatsapp** pengguna / pembeli. Nomor yang digunakan bisa diawali dengan “08” dan “+62”, jadi pesan dapat dikirimkan walaupun diawali “+62”

```java
package shop.notification;

import java.util.ArrayList;
import java.util.List;

public class NotificationService {

    private List<NotificationChannel> daftarChannel = new ArrayList<>();

    public void register(NotificationChannel channel) {
        daftarChannel.add(channel);
    }

    public int broadcast(CustomerContact contact, String message) {

        int totalSend = 0;

        for (NotificationChannel channel : daftarChannel) {

            if (channel.send(contact, message)) {

                totalSend++;
            }
        }
        return totalSend;
    }
```

Class ini yang akan mendaftarkan pesan channel tersebut kedalam sebuah list channel. Lalu, **broadcast()** mengembalikan lokal variable **totalSend** yang bertambah setiap pesan channel yang dikirimkan.

3. Membuat class ShopApp.java (main)

```java
package shop.app;

import java.util.List;

import shop.StudentInfo;
import shop.model.Address;
import shop.model.DigitalProduct;
import shop.model.Order;

import shop.model.PhysicalProduct;
import shop.model.Product;
import shop.notification.CustomerContact;
import shop.notification.EmailChannel;
import shop.notification.NotificationChannel;
import shop.notification.NotificationService;
import shop.notification.SmsChannel;
import shop.notification.WhatsAppChannel;
import shop.service.CheckoutService;
import shop.shipping.ExpressShipping;

import shop.shipping.ShippingService;
import shop.payment.EWalletPayment;
import shop.payment.BankTransferPayment;
import shop.payment.CreditCardPayment;
import shop.payment.PaymentMethod;

public class TugasApp {
```

```java
public class TugasApp {

    Run | Debug
    public static void main(String[] args) {

        Product coffee = new PhysicalProduct(sku: "KOPI-001", name: "Kopi Arabika Gayo 250g", price: 85_000, weightGrams: 300);

        Product rendang = new PhysicalProduct(sku: "RDG-002", name: "Rendang Kemasan 250g", price: 120_000, weightGrams: 400);

        Product ebook = new DigitalProduct(sku: "EBK-003", name: "E-book Resep Kue Nusantara", price: 50_000,
                downloadUrl: "https://toko-umkm.example/unduh/ebk-003");

        List<Product> cart = List.of(coffee, rendang, ebook);

        // L2 => interface and program to an interface
        CheckoutService checkout = new CheckoutService();
        // ShippingService regular = new RegularShipping();
        ShippingService express = new ExpressShipping();

        CreditCardPayment card = new CreditCardPayment(cardHolder: "Siti Rahma", creditLimit: 1_000_000);
        EWalletPayment wallet = new EWalletPayment(provider: "DompetKu : ", balance: 300_000);
        PaymentMethod transfer = new BankTransferPayment(bankName: "Bank Nagari");

        long totalExpress = checkout.total(cart, express);
        long totalBank = checkout.grandTotal(cart, express, transfer);
        long totalKredit = checkout.grandTotal(cart, express, card);

        Address address = new Address(street: "Jl. Moh Hatta No. 10", city: "Padang", postalCode: "25121");
        Order order = new Order(orderId: "ORD-0001", address);

        System.out.println("Tugas oleh Razif Al Farisi (2511532028)\n");
```

```java
    System.out.println("Tugas oleh Razif Al Farisi (2511532028)\n");

    System.out.println("=== T1: Total dengan biaya admin(Ekspress)");
    System.out.println("E-Wallet Dompetku   : Rp" + totalExpress);
    System.out.println("Total Bank          : Rp" + totalBank);
    System.out.println("Total Kredit        : Rp" + totalKredit);

    System.out.println("Kebijakan Kartu Kredit  : " + card.policy());
    System.out.println("Kebijakan e-wallet      : " + wallet.policy());


    EmailChannel email = new EmailChannel();
    WhatsAppChannel whatsapp = new WhatsAppChannel();
    SmsChannel sms = new SmsChannel(StudentInfo.SMS_MAX_LENGTH);

    NotificationService daftarchannel = new NotificationService();
    CustomerContact budi = new CustomerContact(name: "Budi01", email: " ", phone: "+621464123252");
    CustomerContact siti = new CustomerContact(name: "Siti Rahma", email: "SitiRahma@mail.co", phone: "081464123252");
    String pesan = "Pesanan " + order.getOrderId()
            + " sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk


    daftarchannel.register(email);
    daftarchannel.register(sms);
    daftarchannel.register(whatsapp);
    daftarchannel.register(new NotificationChannel() {
        @Override
        public String channelName() {
```

```java
        @Override
        public String channelName() {
            return "LOG System";
        }

        @Override
        public boolean send(CustomerContact contact, String message) {
            System.out.println("[LOG] " + contact.getName() + " : " + message);
            return true;
        }
    });

    System.out.println("\n==T2: Notifikasi Status Pesanan==");
    // customer siti
    System.out.println("====Pesan ke Siti====");
    System.out.println("Terkirim ke Siti lewat " + daftarchannel.broadcast(siti, pesan) + " Channel");

    // customer budi
    System.out.println("\n====Pesan Ke Budi01====");
    System.out.println("Terkirim ke Budi lewat " + daftarchannel.broadcast(budi, pesan) + " Channel");
}
```

Semua Objek di buat terlebih dahulu, lalu total harga untuk pengiriman express di buat untuk setiap metode pembayaran yang digunakan. **card.policy()** dan **wallet.policy()** memiliki aturan berbeda(kredit 12 bulan). Terdapat anonymous class dengan kode “daftarChannel.register(new ...)”, diberi tag untuk (LOG), method **send()** juga menampilkan pesan Log dari sistem.

4. Hasilnya seperti berikut

```
Tugas oleh Razif Al Farisi (2511532028)

=== T1: Total dengan biaya admin(Ekspress)
E-Wallet Dompetku    : Rp268000
Total Bank           : Rp272000
Total Kredit         : Rp276040
Kebijakan Kartu Kredit  : Refund maksimal 7 hari; Cicilan maksimal 12 bulan
Kebijakan e-wallet      : Refund maksimal 7 hari

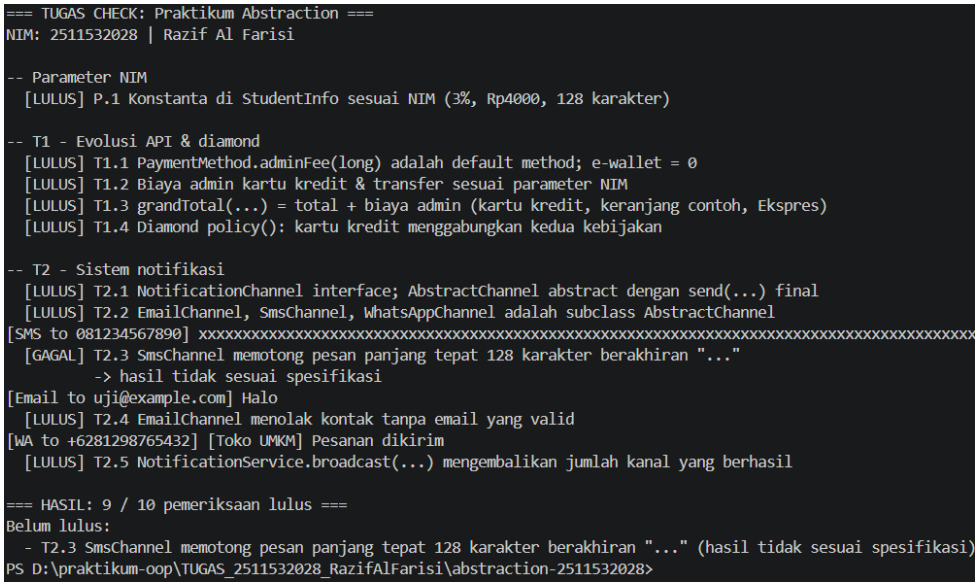
==T2: Notifikasi Status Pesanan==
====Pesan ke Siti====
[Email to SitiRahma@mail.co] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja pr
oduk UMKM Sumatera Barat di toko kami!
[SMS to 081464123252] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbe...
[WA to 081464123252] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanj
a produk UMKM Sumatera Barat di toko kami!
[LOG] Siti Rahma : Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM
Sumatera Barat di toko kami!
Terkirim ke Siti lewat 4 Channel

====Pesan Ke Budi01====
[WA to +621464123252] [Toko UMKM] Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelan
ja produk UMKM Sumatera Barat di toko kami!
[LOG] Budi01 : Pesanan ORD-0001 sudah dikirim dengan kurir Ekspres. Nomor resi akan muncul di aplikasi dalam 1x24 jam. Terima kasih telah berbelanja produk UMKM Suma
tera Barat di toko kami!
Terkirim ke Budi lewat 2 Channel
PS D:\praktikum-oop\TUGAS_2511532028_RazifAlfarisi\abstraction-2511532028>
```

5. Jawab pertanyaan :
   - Interface berfungsi sebagai kontrak perjanjian bahwa kanal notifikasi harus bisa me-*return* nilai pengiriman, sedangkan abstract class digunakan untuk menampung kode yang pasti terpakai berulang kali oleh semua kanal (seperti pengecekan *null* dan penghitung pesan).
   - Subclass bisa saja menimpa (override) method tersebut lalu sengaja menghapus logika pengecekan penerima atau melupakan penambahan counter, yang ujung-ujungnya merusak rekam jejak (*log*) pengiriman seluruh sistem.

- Fleksibilitas, kalau kita pakai tipe interface, kita siap menerima penambahan kanal apa pun di masa depan termasuk class aneh-aneh buatan pihak ketiga yang tidak *extends* dari AbstractChannel kita, asalkan mereka tetap mematuhi aturan kontrak NotificationChannel.

6. TugasCheck


=== TUGAS CHECK: Praktikum Abstraction ===
NIM: 2511532028 | Razif Al Farisi
-- Parameter NIM
[LULUS] P.1 Konstanta di StudentInfo sesuai NIM (3%, Rp4000, 128 karakter)
-- T1 - Evolusi API & diamond
[LULUS] T1.1 PaymentMethod.adminFee(long) adalah default method; e-wallet = 0
[LULUS] T1.2 Biaya admin kartu kredit & transfer sesuai parameter NIM
[LULUS] T1.3 grandTotal(...) = total + biaya admin (kartu kredit, keranjang contoh, Ekspres)
[LULUS] T1.4 Diamond policy(): kartu kredit menggabungkan kedua kebijakan
-- T2 - Sistem notifikasi
[LULUS] T2.1 NotificationChannel interface; AbstractChannel abstract dengan send(...) final
[LULUS] T2.2 EmailChannel, SmsChannel, WhatsAppChannel adalah subclass AbstractChannel
[SMS to 081234567890] xxxx

perlu mengakses cart terlebih dahulu.(tambahkan static pada class item).

- Pada GiftVoucher.balance, tipenya double untuk uang tersebut. Resikonya output dari kode tersebut berkoma(20000.0) atau saat menjalankan pay(-5000) malah bertambah saldo.(ganti menjadi long & validasi amount > 0).

8. Isi tabel berikut :

No.	(class / method)	Masalah	Dampak	Perbaikan
1	GiftVoucher.refund() & ship()	Method tersebut tidak mendukung.	CrashSaat runtime: 	Menghapus ship() / menambahkan ke class yang butuh ship()
2	Courier, cost()	Tidak memiliki abstract method & cost bisa di ubah(final)	Kurir tidak dipaksa menentikan tarif sendiri, dan aturan validasi bisa diubah	Tambah protected abstract long ratePerKg(). Tambah final pada method cost()
3	Address	Mengubah satu alamat order, yang terubah semua alamat order	Yang diubah city dari id ORD-2, tapi yang ke apply ORD-1 dan ORD-2 (bukittinggi)	Final class Address.
4	Cart.Item	Inner class tapi tidak ada static, jadi harus mengakses ke cart dulu.	Referensi tersembunyi di cart dan pembuatan harus lewat cart.new Item....	Jadikan static final class item.
5	GiftVoucher.balance	Tipe double dan tidak ada validasi	Output 70000.0 dan jika menjalankan pay(-5000) yang ada malah menambah saldo	Ganti menjadi long, saldo awal lewat constructor, validasi amount > 0.

6	Discount.apply	Melanggar OCP dan mematikan polimorfisme	Diskon yang baru ditambahkan tidak berlaku karena jatuh ke return price	Jadikan apply() abstract, persentase ditentukan tiap implementasi

9. Implementasi ke fixed.BadShopRefactored.java

```java
// perbaikan GiftVoucher unsupportedOperation
interface StoreOperations {
    boolean pay(long amount);

    boolean refund(long amount);

    String printInvoice();
}
```

```java
class GiftVoucher implements StoreOperations {
    private long balance;
    // perbaikan GiftVoucher
    public GiftVoucher(long balance) {
        if (balance < 0) {
            throw new IllegalArgumentException("Saldo awal tidak boleh negatif");
        }
        this.balance = balance;
    }

    @Override
    public boolean pay(long amount) {
        if (amount <= 0 ) {
            throw new IllegalArgumentException("Jumlah bayar harus lebih dari 0");
        }
        if (amount > balance) {
            return false;
        }
        balance -= amount;
        return true;
    }
```

Perbaikan pada balance dan pemeriksaan amount < 0 agar tidak terjadi kesalahan input.

```java
@Override
public boolean refund(long amount) {
    if (amount > balance) {
        return false;
    }
    balance += amount;
    return true;
}

@Override
public String printInvoice() {
    return "Voucher, remaining balance " + balance;
}
```

Ship() di hapus dan refund diisi pada class GiftVoucher.

```java
// perbaikan discount, percentdiscount, member, dan flashsale
interface Discount {
    long apply(long price);
}

abstract class PercentDiscount implements Discount {
    private final int percent;

    protected PercentDiscount(int percent) {
        this.percent = percent;
    }

    @Override
    public final long apply(long price) {
        return price - price * percent / 100;
    }
```

```java
    @Override
    public final long apply(long price) {
        return price - price * percent / 100;
    }
}

class MemberDiscount extends PercentDiscount {
    MemberDiscount() {
        super(percent: 5);
    }
}

class FlashSaleDiscount extends PercentDiscount {
    FlashSaleDiscount() {
        super(percent: 30);
    }
}
```

Class discount dipecah menjadi beberapa class seperti percentDiscount, member, dan flashsale.

10. Tabel 1

Pertanyaan desain	Abstract / Interface?	Alasan	Contoh di proyek
Subclass berbagi field dan konstruktor	**Abstract class**	Interface tidak punya field instance (hanya konstanta) dan tidak punya konstruktor. Abstract class bisa menyimpan state bersama dan mengisinya lewat konstruktor.	PercentDiscount punya field percent dan konstruktor PercentDiscount(int percent). MemberDiscount dan FlashSaleDiscount cukup memanggil super(5) dan super(30).
Satu class perlu beberapa kemampuan sekaligus	**Interface**	Java hanya mengizinkan satu extends, tetapi satu class boleh implements banyak interface.	CashPayment implements Payable, Refundable dan GiftVoucher implements Payable, Printable. Voucher hanya mengambil kemampuan yang dia punya.
Rumus bersama yang tidak boleh diubah subclass	**Abstract class**	Abstract class boleh punya method konkret yang final (*template method*). Method interface tidak bisa final, jadi tidak ada jaminan rumus tidak ditimpa.	Courier.cost() bersifat final (validasi dan perkalian tarif × berat), sedangkan tarif diserahkan ke ratePerKg() yang abstract. PercentDiscount.apply() juga final.
Kontrak yang perlu ditambah method baru di kemudian hari	**Interface** (dengan *default method*)	Menambah method abstract ke interface akan merusak semua implementor. Dengan default, method baru langsung punya isi bawaan sehingga kode lama tetap bisa dikompilasi.	Discount bisa ditambah default String label() tanpa mengubah MemberDiscount dan FlashSaleDiscount. Perhatikan: Discount.apply() di BadShop.java asli adalah **penyalahgunaan** default method karena memakai instanceof. Pemakaian yang benar adalah untuk perilaku umum yang netral, bukan untuk mengecek subclass.

11. Tabel 2

Kondisi	Static nested, inner, atau anonymous?	alasan	Contoh di proyek
Class pembantu tidak membaca data objek luar	**Static nested**	Tidak menyimpan referensi tersembunyi ke objek luar, bisa dibuat tanpa instance Cart, dan lebih hemat memori.	Cart.Item hanya memakai field miliknya (name, price, quantity), jadi seharusnya static. Di kode asli ia inner (cart.new Item(...)), salah. Di refactor menjadi static final class Item, dibuat dengan new Cart.Item(...).
Objek harus membaca field objek luarnya	**Inner class** (non-static)	Inner class otomatis memegang referensi ke objek luar sehingga bisa langsung mengakses field-nya, termasuk yang private.	Proyek ini sebenarnya tidak punya contoh yang benar-benar butuh. Contoh jika ditambahkan ke Cart: class ItemIterator implements Iterator<Item> yang membaca list items milik Cart
Implementasi sekali pakai, tidak perlu nama	**Anonymous class**	Implementasi dibuat dan dipakai di satu tempat, jadi tidak perlu mendeklarasikan class lagi.	Diskon sekali pakai untuk Discount di main: new Discount() { public long apply(long p) }

Link Repository

https://github.com/razif05/PrakPBO_B_2511532028/blob/Praktikum-6/abstraction-2511532028